# jssrepl

## User Manual

An interactive JavaScript shell for GraalVM, in the spirit of the R and Scala REPLs.

### Contents

1. Introduction
2. Starting jssrepl
3. Using the shell
4. Line editing and history
5. Shell commands
6. Libraries
7. Parameters
8. Java interop
9. Sample sessions
10. Known limitations
11. Troubleshooting

# 1. Introduction

jssrepl is a small interactive JavaScript shell built on GraalJS. You type JavaScript, jssrepl evaluates it, and the result is printed. State persists between lines, so variables and functions you define stay available for the rest of the session.

Library files given on the command line are loaded first, and the shell then stays open so you can keep working with what they defined. This matches `scala -i lib.scala` or sourcing a file in an R session. jssrepl never exits unless you ask it to.

## Key features

- Multi-line input: unfinished brackets or strings automatically continue on a new prompt.
- Readable output: strings are quoted, objects are pretty-printed as JSON, functions show their source.
- Line editing with Left/Right arrows, plus Up/Down recall of your last 6 commands.
- Library preloading, and a `parameters` array and `params` object for command-line arguments.
- Full Java interop through GraalVM.
- Utility commands: `:load`, `:vars`, `:time`, `:history`, `:reset`.

# 2. Starting jssrepl

Run the jar with Java. All forms below assume `jss.jar` and its dependencies (GraalVM polyglot and JLine) are available to your Java runtime.

| Command | Effect |
|---|---|
| `java -jar jss.jar` | Start the REPL with nothing preloaded. |
| `java -jar jss.jar lib1.js lib2.js` | Load lib1.js, then lib2.js, then stay in the REPL. |
| `java -jar jss.jar scriptsPath=/tmp/foo` | Start the REPL. The argument is not a file, so it is only available as **parameters[1]** and **params.scriptsPath**. |
| `java -jar jss.jar -e "1 + 1"` | Evaluate one expression, print the result, and exit. |
| `java -jar jss.jar -h` | Show usage and exit. (--help also works.) |

## Start-up banner

In interactive mode jssrepl prints a short banner, then the `js>` prompt:

```
jss - GraalJS interactive shell
Type JavaScript to evaluate it, or :help for commands.
Left/Right arrows edit the line; Up/Down browse the last 6 commands.
To see parameters when you ran jssrepl, type:parameters, it is a JavaScript array you can use.
jssrepl home directory is stored in parameters array.
js>
```

## One-shot evaluation

The `-e` option is handy in shell scripts:

```
$ java -jar jss.jar -e "[1,2,3].map(x => x * 2)"
[
  2,
  4,
  6
]
```

**Note.** In `-e` mode, **parameters** and **params** are not defined, and library files are not loaded. Use the interactive form when you need them.

# 3. Using the shell

Type any JavaScript expression or statement at the `js>` prompt and press Enter. The result is printed on the next line.

## How results are displayed

| Value type | Display |
|---|---|
| Numbers, booleans | Printed as-is, for example **42**, **true**. |
| Strings | Printed in double quotes (JSON style), for example **"hello"**. |
| Objects and arrays | Pretty-printed as JSON with 2-space indentation. |
| Functions | Printed as their source text. |
| null | Printed as **null**. |
| undefined | **Not printed.** Statements such as var declarations or calls to print() would otherwise add noise. To see it, wrap it: **(undefined)**. |

```
js> 6 * 7
42
js> "hello".toUpperCase()
"HELLO"
js> var user = { name: "Ada", langs: ["js", "java"] }
js> user
{
  "name": "Ada",
  "langs": [
    "js",
    "java"
  ]
}
js> const square = x => x * x
js> square
x => x * x
js> square(12)
144
```

## Multi-line input

If brackets, strings, template literals or block comments are still open when you press Enter, jssrepl shows the continuation prompt `...>` and keeps reading. The statement runs once everything is closed.

```
js> function greet(name) {
  ...> return "Hello, " + name + "!";
  ...> }
js> greet("Ada")
"Hello, Ada!"
```

## Errors

Runtime and syntax errors are reported on one line, and the session continues with its state intact.

```
js> foo
Error: ReferenceError: foo is not defined
js> )
SyntaxError: unexpected closing bracket
```

> **Note.** The exact wording of JavaScript error messages comes from GraalJS and may differ slightly between versions.

## Cancelling input

Press **Ctrl-C** to abandon the line, or multi-line statement, you are typing. jssrepl prints `^C` and returns to a fresh prompt. Press **Ctrl-D** on an empty prompt to exit.

# 4. Line editing and history

Line editing is provided by JLine. jssrepl keeps its own rolling history of your last **6** submitted commands.

| Key | Action |
|---|---|
| Left / Right | Move the cursor within the current line (standard JLine behavior). |
| Up | Show the previous (older) command. From the oldest command, Up wraps around to the newest. |
| Down | Show the next (newer) command. From the newest command, Down clears the line. |
| Ctrl-C | Abandon the current input. |
| Ctrl-D | Exit the shell. |

## Recall rules

- Recalling a command always replaces whatever is currently typed on the line.
- If nothing has been entered yet, Up and Down do nothing.
- Only the last 6 commands are kept; older ones drop off. Submitting a command ends recall mode.
- Use `:history` to list the saved commands.

```
js> 1 + 1
2
js> 2 + 2
4
js> :history
[1] 1 + 1
[2] 2 + 2
```

Pressing Up now shows `2 + 2`, pressing Up again shows `1 + 1`, and pressing Up a third time wraps back to `2 + 2`. Pressing Down from the newest command gives a blank line.

> **Note.** `:`commands are not stored in the history, only JavaScript snippets are.

# 5. Shell commands

Lines beginning with a colon are shell commands rather than JavaScript. They are recognized only at a fresh `js>` prompt, not in the middle of a multi-line statement.

| Command | Description |
|---|---|
| `:help, :h, :?` | Show the command summary. |
| `:load <file...>, :l` | Load and run one or more .js files into the current session. |
| `:vars` | List the top-level bindings you have created this session, with their values. |
| `:time <expr>` | Evaluate an expression, print the result, then print the elapsed time. |
| `:history` | Show the last 6 evaluated snippets, numbered. |
| `:reset` | Discard all state and start a fresh JavaScript context. Also clears history. |
| `:quit, :exit, :q` | Exit the shell (Ctrl-D also works). |

### :vars

```
js> var count = 3
js> var label = "items"
js> :vars
count = 3
label = "items"
```

**Note.** `:vars` lists names registered as global bindings. Declarations made with `var` and `function` are the most reliable to see here. If a `let` or `const` name is missing from the list, it is still usable in the session.

### :time

```
js> :time (function(){ var s = 0; for (var i = 0; i < 1e6; i++) s += i; return s; })()
499999500000
(12.481 ms)   <- example timing; yours will differ
```

### :reset

```
js> var x = 10
js> :reset
Context reset.
js> x
Error: ReferenceError: x is not defined
```

**Note.** After `:reset`, the **parameters** and **params** values and any libraries loaded at start-up are gone. Reload libraries with `:load`.

# 6. Libraries

A library is just a JavaScript file. Files named on the command line are loaded before the prompt appears; `:load` loads more at any time. Everything the file defines at the top level is then available in the session.

## Example library: mathlib.js

```
// mathlib.js
function factorial(n) {
  return n <= 1 ? 1 : n * factorial(n - 1);
}

function fib(n) {
  var a = 0, b = 1;
  for (var i = 0; i < n; i++) { var t = a + b; a = b; b = t; }
  return a;
}

function mean(xs) {
  return xs.reduce(function (s, x) { return s + x; }, 0) / xs.length;
}
```

## Loading at start-up

```
$ java -jar jss.jar mathlib.js
Loaded: mathlib.js
jss - GraalJS interactive shell
...
js> factorial(5)
120
js> fib(10)
55
js> mean([1, 2, 3, 4])
2.5
```

## Loading from inside the shell

```
js> :load mathlib.js stats.js
Loaded: mathlib.js
Loaded: stats.js
js> :load missing.js
File not found: missing.js
js> :load /tmp
The parameter is a directory: /tmp
To run a jss file do: :load <path-to-jss-file>
```

If a file's last statement produces a value, it is shown after the file name, for example `Loaded: config.js => {...}`. A file made only of declarations shows just the file name.

> **Note.** A file that throws an error is reported as an error, and loading stops. Definitions made before the error remain in the session.

# 7. Parameters

Inside the interactive shell, jssrepl always defines two values for you:

- **parameters** is a JavaScript array. **parameters[0]** is always the jssrepl home directory (where jss.jar lives). **parameters[1]**, **parameters[2]**, ... are the arguments you passed.
- **params** is an object built from any argument shaped like **key=value**. Arguments without an equals sign are skipped here, but still appear in **parameters**.

```
$ java -jar jss.jar scriptsPath=/tmp/scripts verbose=true
...
js> parameters.length
3
js> parameters[1]
"scriptsPath=/tmp/scripts"
js> params.scriptsPath
"/tmp/scripts"
js> params
{
  "scriptsPath": "/tmp/scripts",
  "verbose": "true"
}
```

All values in **params** are strings. Convert them yourself when needed, for example `params.verbose === "true"` or `Number(params.limit)`.

## Using parameters with libraries

An argument that is an existing file is loaded as a library, and is also present in **parameters**. A library can therefore read its own configuration:

```
// loader.js - run as: java -jar jss.jar loader.js dir=/data
var dataDir = params.dir || "/tmp";
```

> **Note.** Libraries are loaded after **parameters** and **params** are defined, so they can use both.

# 8. Java interop

jssrepl runs with full host access, so JavaScript can use Java classes through `Java.type`.

```
js> var ArrayList = Java.type('java.util.ArrayList')
js> var list = new ArrayList()
js> list.add("alpha")
true
js> list.add("beta")
true
js> list.size()
2
js> list.get(1)
"beta"
js> var System = Java.type('java.lang.System')
js> System.getProperty('java.version')
"21.0.2"
```

The version string above is only an example. Your value depends on the Java runtime you started jssrepl with.

> **Note.** Full host access means scripts you load can read and write files and run code on your machine, just like any Java program. Only load libraries you trust.

# 9. Sample sessions

## Session A: quick calculations and data

```
js> var prices = [19.99, 5.25, 3.5]
js> var total = prices.reduce((a, b) => a + b, 0)
js> total.toFixed(2)
"28.74"
js> prices.map(p => Math.round(p))
[
  20,
  5,
  4
]
js> JSON.parse('{"ok": true}').ok
true
```

## Session B: build up a small library interactively

```
js> function isPrime(n) {
  ...>   if (n < 2) return false;
  ...>   for (var i = 2; i * i <= n; i++) if (n % i === 0) return false;
  ...>   return true;
  ...> }
js> Array.from({length: 20}, (_, i) => i + 1).filter(isPrime)
[
  2,
  3,
  5,
  7,
  11,
  13,
  17,
  19
]
js> :time isPrime(1000003)
true
(0.412 ms)
```

## Session C: a library and parameters together

Create `tools.js`:

```
function shout(s) { return s.toUpperCase() + "!"; }
var greeting = "Hello from " + (params.name || "nobody");
```

Run it with a parameter:

```
$ java -jar jss.jar tools.js name=Ada
Loaded: tools.js  => "Hello from Ada"
jss - GraalJS interactive shell
...
js> shout("done")
"DONE!"
js> greeting
```

```
"Hello from Ada"
js> :vars
greeting = "Hello from Ada"
params = {
  "name": "Ada"
}
parameters = [
  "/opt/jssrepl",
  "tools.js",
  "name=Ada"
]
```

**Note.** The home directory **/opt/jssrepl** and the order of items in `:vars` are examples and will differ on your system.

# 10. Known limitations

jssrepl is intentionally small. Be aware of these behaviors in the current version:

- **Multi-line detection is approximate.** jssrepl counts brackets and tracks strings and comments, but it is not a full parser. Regular expression literals containing quotes or brackets, such as `/"/` or `/[(]/`, can confuse it and leave you at the `...>` prompt. Press Ctrl-C to recover.
- **Template literals:** brackets inside `${ }` are not tracked.
- **History is small and temporary.** Only the last 6 commands are kept, and they are not saved between sessions.
- **Multi-line recall:** recalled multi-line snippets keep their line breaks inside the edit buffer.
- **Ctrl-C cannot stop a running evaluation,** such as an infinite loop. It only cancels input being typed. Stop a runaway script by terminating the Java process from another terminal.
- **-e mode** does not define **parameters** or **params**.
- **Special characters in arguments:** on Windows, an argument containing a colon after the equals sign, for example `path=C:\tmp`, may be mistaken for an invalid file path at start-up.

# 11. Troubleshooting

| Symptom | Likely cause and fix |
|---|---|
| The prompt stays at ...> after a line that looks complete | An opening bracket, quote, or comment is unclosed, or a regex literal confused the tracker. Press Ctrl-C and retype. |
| Nothing is printed after a statement | The result was **undefined**, which jssrepl hides. Use **(undefined)** if you want to see it. |
| Up/Down does nothing | No commands have been entered yet in this session, or **:reset** just cleared the history. |
| Unknown command: :foo (type :help) | Only the commands in section 5 exist. Check the spelling. |
| File not found: ... | The path given to **:load** is wrong. Relative paths are resolved from the directory where you launched Java. |
| Internal error: ... | The error came from the host (Java) side rather than your JavaScript. Check Java interop calls and class names. |
| parameters is not defined | You are in -e mode, or you ran **:reset**. Use the interactive shell and avoid :reset if you need it. |
| A variable has disappeared | **:reset** discards all state. Reload your libraries with **:load**. |

## Quick reference

```
:help            show commands          :vars       list your top-level bindings
:load f.js       load a library         :time expr  evaluate and time an expression
:history         last 6 commands        :reset      start a fresh context
:quit            exit (or Ctrl-D)

parameters[0]    jssrepl home directory   parameters[1..]  your arguments
params.key       value of key=value args  Java.type('...')  use any Java class
```